

CRAFT: Supplementary Experimental Analyses

This supplement reports analyses that are useful for assessing robustness and systems cost but are not required for the main argument. Appendix A tests whether duplicate instances within the evaluation benchmark change the conclusions of the Qwen3-8B training experiments. Appendix B characterizes the computational profile of the evaluation used for Qwen3-8B and all of its training variants. All verification rates are percentages.

A Robustness to Benchmark Deduplication

A.1 Question and protocol

The full benchmark contains 832 programs. A separate audit identifies 124 evaluation instances as duplicates of other instances within this benchmark; removing those redundant instances leaves 708 programs. This is an evaluation-set deduplication analysis, distinct from the training–evaluation overlap audit reported in the main paper. We use the reduced set as a sensitivity analysis rather than as a replacement benchmark. The model checkpoints, saved response order, sampling budgets $k \in \{1, 4, 8, 16, 32\}$, composition procedure, and verifier are unchanged. Thus, differences below reflect the changed evaluation population, not retraining or resampling.

We report two complementary comparisons. The *training-stage comparison* follows representative Bare, RL-from-Bare, and SFT+RL checkpoints. The *matched reward ablation* separately compares Binary, Whole-rollout, Clause-decomposed, and Full reward runs within a common initialization. These are distinct evaluation groups and should not be cross-compared as if they were the same checkpoint.

A.2 Training-stage comparison

Tables 1 and 2 pair the full and internally deduplicated evaluations. Parenthesized entries on the 708-program rows are percentage-point changes from the corresponding 832-program result.

Table 1: Qwen3-8B training-stage comparison under pass@ k .

Setting	N	@1	@4	@8	@16	@32
Bare	832	8.27	16.45	20.53	24.36	28.12
	708	7.97 (-0.30)	15.87 (-0.58)	19.73 (-0.81)	23.29 (-1.07)	26.84 (-1.29)
RL from Bare	832	2.88	5.41	7.81	9.62	11.06
	708	2.54 (-0.34)	4.24 (-1.17)	6.64 (-1.17)	7.91 (-1.71)	9.18 (-1.88)
SFT+RL	832	32.93	46.88	51.44	53.37	55.89
	708	30.37 (-2.57)	43.64 (-3.23)	47.88 (-3.56)	49.72 (-3.65)	52.40 (-3.49)

Table 2: Qwen3-8B training-stage comparison under compose@ k .

Setting	N	@1	@4	@8	@16	@32
Bare	832	43.75	55.17	59.01	61.18	62.50
	708	42.09 (-1.66)	53.81 (-1.35)	57.91 (-1.10)	60.17 (-1.01)	61.44 (-1.06)
RL from Bare	832	51.80	60.46	64.18	68.87	70.67
	708	48.87 (-2.93)	57.20 (-3.25)	61.16 (-3.02)	65.68 (-3.19)	67.51 (-3.16)
SFT+RL	832	66.23	72.48	74.40	75.12	75.60
	708	62.99 (-3.23)	69.07 (-3.41)	71.05 (-3.35)	71.89 (-3.23)	72.32 (-3.28)

Stage conclusions. Removing within-benchmark duplicates lowers the reported rates but preserves the shape of both curves. Across the two tables, the absolute changes range from 0.30 to 3.65 points. The larger shifts for SFT+RL indicate that the removed subset is comparatively favorable to that checkpoint, but SFT+RL remains the strongest stage at every budget under both pass and composition. Likewise, RL from Bare continues to improve compose@ k over Bare while reducing pass@ k , preserving the distinction between standalone-response quality and cross-response clause utility.

A.3 Stability of the reward ablation

The central robustness question is whether internal benchmark deduplication changes the reward ranking. Tables 3 and 4 report the complete matched ablation. Each reward has adjacent 832- and 708-program rows. “Bare (reference)” appears only in the Bare-initialization panel; it is not an additional trained reward.

Table 3: Reward ablation under compose@ k . The two panels distinguish RL directly from Bare from RL after SFT.

Reward	N	@1	@4	@8	@16	@32
<i>RL directly from Bare</i>						
Bare (reference)	832	43.75	55.17	59.01	61.18	62.50
	708	42.09	53.81	57.91	60.17	61.44
Binary	832	0.00	0.00	0.00	0.00	0.00
	708	0.00	0.00	0.00	0.00	0.00
Whole-rollout	832	20.79	21.88	22.24	22.84	23.32
	708	19.63	20.76	21.19	21.61	22.03
Clause-decomposed	832	48.80	57.69	60.70	63.70	66.35
	708	46.75	55.93	59.04	61.86	64.69
Full (ours)	832	51.80	60.46	64.18	68.87	70.67
	708	48.87	57.20	61.16	65.68	67.51
<i>RL after SFT</i>						
Binary	832	7.09	10.82	13.34	14.54	15.99
	708	6.78	10.03	12.57	13.84	15.54
Whole-rollout	832	48.80	56.37	60.22	63.10	66.59
	708	47.03	53.67	57.06	59.89	63.42
Clause-decomposed	832	67.91	75.84	76.80	77.64	78.25
	708	64.69	72.88	73.73	74.72	75.14
Full (ours)	832	68.51	75.72	77.52	78.25	78.37
	708	64.83	72.46	74.58	75.28	75.42

Table 4: Reward ablation under $\text{pass}@k$. Pass measures whether any complete response verifies alone and therefore need not follow the compose ranking.

Reward	N	@1	@4	@8	@16	@32
<i>RL directly from Bare</i>						
Bare (reference)	832	8.27	16.45	20.53	24.36	28.12
	708	7.97	15.87	19.73	23.29	26.84
Binary	832	4.45	4.45	4.45	4.45	4.45
	708	4.52	4.52	4.52	4.52	4.52
Whole-rollback	832	15.50	16.59	16.83	17.67	18.39
	708	15.54	16.67	16.95	17.66	18.08
Clause-decomposed	832	9.01	15.87	18.87	20.31	21.63
	708	8.19	14.69	17.66	19.21	20.62
Full (ours)	832	2.88	5.41	7.81	9.62	11.06
	708	2.54	4.24	6.64	7.91	9.18
<i>RL after SFT</i>						
Binary	832	8.77	11.30	13.22	14.30	15.50
	708	8.47	10.73	12.57	13.70	15.11
Whole-rollback	832	41.11	49.28	53.12	55.53	58.29
	708	38.56	46.05	49.58	51.84	54.66
Clause-decomposed	832	35.94	51.44	54.33	57.69	61.30
	708	32.06	47.32	50.56	53.39	56.78
Full (ours)	832	36.78	51.92	56.49	59.50	61.54
	708	32.91	47.88	52.40	55.23	57.34

The composition result is robust. Clause-level rewards dominate response-level rewards under $\text{compose}@k$ on both evaluation sets and from both initializations. At $\text{compose}@32$ the strict ordering is unchanged:

$$\text{Full} > \text{Clause-decomposed} > \text{Whole-rollback} > \text{Binary}.$$

After SFT, Full and Clause-decomposed are close: Clause-decomposed is slightly higher at @4, whereas Full is higher at @1, @8, @16, and @32 on both sets. These small local crossings do not alter the endpoint ranking or the larger gap between clause-level and response-level credit.

Pass and compose expose different effects. Whole-rollback can retain stronger $\text{pass}@k$ than the clause-level objectives, especially from Bare, while performing substantially worse under composition. This is consistent with the objectives: response-level credit favors complete standalone answers, whereas clause-level credit preserves useful partial content for later composition. The within-benchmark deduplication control retains this contrast.

A.4 Negative-trace sampling ablation

We next isolate negative-trace sampling while holding the Full reward fixed. Structure uses the target-hidden negative-trace construction defined in the main paper; Random replaces it with a budget-matched random construction. We compare both samplers under RL and SFT+RL. As in the main paper, RL denotes training directly from Bare, whereas SFT+RL starts from the SFT model.

Tables 7 and 8 disaggregate the same comparison by program class. The “All” results are reported separately above to avoid repeating the aggregate in the stratified tables.

Negative-trace sampling mainly affects standalone success. On the full benchmark, Structure improves $\text{pass}@k$ over Random at every response budget: the gain ranges from 0.72 to 3.61 points under SFT+RL and from 1.32 to 2.52 points under RL. The stratified results locate most of this gain in Linear and Loopy programs. The small NLA subset changes by at most two points, corresponding to a single program, and often ties.

Table 5: Qwen3-8B negative-trace sampling ablation on all 832 programs under pass@ k , with the Full reward. Difference rows report Structure minus Random in percentage points.

Training	Sampler	@1	@4	@8	@16	@32
SFT+RL	Structure	37.98	51.80	56.13	59.98	62.74
	Random	37.26	48.20	53.85	58.05	60.70
	Δ	+0.72	+3.61	+2.28	+1.92	+2.04
RL	Structure	8.17	13.10	14.90	17.43	19.35
	Random	5.65	11.18	13.22	15.87	18.03
	Δ	+2.52	+1.92	+1.68	+1.56	+1.32

Table 6: Qwen3-8B negative-trace sampling ablation on all 832 programs under compose@ k , with the Full reward. Difference rows report Structure minus Random in percentage points.

Training	Sampler	@1	@4	@8	@16	@32
SFT+RL	Structure	67.55	76.08	77.52	78.61	78.73
	Random	68.39	75.24	77.16	78.25	78.49
	Δ	-0.84	+0.84	+0.36	+0.36	+0.24
RL	Structure	49.28	57.93	62.26	64.90	66.83
	Random	50.48	59.86	62.50	64.78	66.11
	Δ	-1.20	-1.93	-0.24	+0.12	+0.72

Table 7: Pass@ k negative-trace sampling ablation by program class.

Training	Sampler	@1	@4	@8	@16	@32
<i>Linear</i> ($n = 316$)						
SFT+RL	Structure	40.51	56.33	59.18	64.56	67.09
	Random	37.66	49.68	56.01	59.49	63.61
RL	Structure	10.13	13.29	15.19	18.35	20.25
	Random	8.23	13.29	15.19	17.41	19.62
<i>NLA</i> ($n = 50$)						
SFT+RL	Structure	6.00	6.00	8.00	8.00	8.00
	Random	6.00	8.00	8.00	8.00	8.00
RL	Structure	0.00	0.00	2.00	2.00	2.00
	Random	0.00	0.00	2.00	2.00	2.00
<i>Loopy</i> ($n = 466$)						
SFT+RL	Structure	39.70	53.65	59.23	62.45	65.67
	Random	40.34	51.50	57.30	62.45	64.38
RL	Structure	7.73	14.38	16.09	18.45	20.60
	Random	4.51	10.94	13.09	16.31	18.67

Table 8: Compose@ k negative-trace sampling ablation by program class.

Training	Sampler	@1	@4	@8	@16	@32
<i>Linear</i> ($n = 316$)						
SFT+RL	Structure	69.94	79.11	80.70	81.33	81.33
	Random	72.47	78.48	80.70	81.33	81.33
RL	Structure	53.16	62.97	66.77	70.25	71.20
	Random	52.53	62.34	65.19	66.77	68.99
<i>NLA</i> ($n = 50$)						
SFT+RL	Structure	12.00	14.00	14.00	14.00	14.00
	Random	12.00	12.00	12.00	12.00	12.00
RL	Structure	10.00	12.00	12.00	12.00	12.00
	Random	12.00	12.00	12.00	14.00	14.00
<i>Loopy</i> ($n = 466$)						
SFT+RL	Structure	71.89	80.69	82.19	83.69	83.91
	Random	71.67	79.83	81.76	83.26	83.69
RL	Structure	50.86	59.44	64.59	66.95	69.74
	Random	53.22	63.30	66.09	68.88	69.74

Composition is insensitive to negative-trace sampling. For $\text{compose}@k$, the aggregate Structure–Random difference lies between -1.93 and +0.84 points and changes sign with the training stage and response budget. The class-level results show the same mixed pattern: neither sampler dominates uniformly across Linear, NLA, and Loopy programs. Thus, Structure provides a modest and consistent benefit for standalone $\text{pass}@k$, but has no systematic effect on $\text{compose}@k$. This ablation also separates negative-trace sampling from the larger reward-design effects above: the qualitative advantage of clause-level credit under composition cannot be attributed to the sampling strategy.

B Cost and Systems Profile of Qwen3-8B and Its Training Variants

B.1 Measurement scope

We profile the common evaluation configuration used for the Qwen3-8B base model and all of its training variants. Each checkpoint is evaluated on all 832 programs with eight rollouts per task. The reported totals are therefore the cost of one complete checkpoint evaluation, not the sum across all variants. The measurement boundary includes prompt processing, response generation, candidate extraction and pooling, Houdini filtering, and final Frama-C/WP verification. Token totals, verifier timings, and memory measurements are recorded quantities. Multi-GPU generation time and the 80-worker wall-clock decomposition use the measured throughput and aggregate work, and therefore describe this serving configuration rather than a hardware-independent latency guarantee.

B.2 Generation workload

Table 9: Token consumption for one Qwen3-8B checkpoint or training-variant evaluation over 832 programs. Each task uses eight rollouts; per-task values are rounded to the nearest token.

Component	Mean per task	Total over 832 tasks
Prompt tokens	1,113	925,635
Output tokens (8 rollouts)	11,057	9,199,244
Total tokens	12,169	10,124,879

Each rollout produces 1,382 output tokens on average and yields approximately 19.6 parsed clauses. Pooling and deduplicating the first eight rollouts leaves 58.8 candidate invariants per task on average, with a maximum of 106. Thus, candidate deduplication removes substantial textual repetition before formal verification: eight responses yield roughly 157 parsed clauses in total, but only 58.8 distinct candidates enter the pooled set on average.

Table 10: Measured vLLM output throughput and derived generation time.

Configuration	Throughput or time
A800-80G throughput	5,833 output tokens/s/GPU
One-GPU generation	26.3 min
Four-GPU generation	6.6 min

The one-GPU time follows from the 9,199,244 generated tokens and measured output-token throughput. The four-GPU value assumes even task allocation at the same per-GPU throughput. Generation is therefore inexpensive relative to the subsequent proof workload once multiple GPUs are available.

B.3 Formal verification dominates compute

Table 11: Frama-C/Houdini verification cost overall and by benchmark subset.

Subset	N	Mean	Median	P95	CPU time	Budget hits
All	832	123.4 s	25.5 s	551.1 s	28.5 h	55 (6.6%)
Linear	316	112.7 s	24.6 s	530.7 s	9.9 h	—
NLA	50	292.0 s	252.6 s	866.8 s	4.1 h	—
Loopy	466	112.6 s	22.3 s	564.5 s	14.6 h	—

Verification time is strongly right-skewed: the overall median is 25.5 seconds, while the P95 is 551.1 seconds and 55 tasks hit the 300-second proof budget. NLA is the most expensive subset per task. Although it contains only 50 programs, it consumes 4.1 CPU-hours and averages 292.0 seconds (4.9 minutes) per program. Loopy consumes the most aggregate CPU time because it is the largest subset, not because its per-task mean exceeds Linear.

B.4 Memory and parallel execution

Table 12: Memory footprint and verifier concurrency.

Component	Memory or concurrency
GPU model weights	15.27 GiB
GPU KV cache	55.43 GiB (capacity: approximately 403K tokens)
Total per GPU	approximately 73.6/80 GiB
CPU per Frama-C worker	approximately 200–500 MB
Concurrent CPU workers	60–100

The 80-GiB GPU is close to capacity, but most allocated memory is the KV cache rather than model weights. On the verifier side, 80 concurrent workers imply approximately 16–40 GB of worker memory from the observed per-process range, excluding the operating system and orchestration overhead. The pipeline is therefore GPU-memory-bound during serving and CPU-parallelism-bound during formal verification.

B.5 End-to-end bottleneck

Table 13: End-to-end wall-clock decomposition with four GPUs and 80 CPU verification workers.

Stage	Wall-clock time	Share
LLM generation	6.6 min	23%
Frama-C verification	21.4 min	77%
Total	approximately 28 min	–

The systems result mirrors the CPU accounting: proof is the bottleneck. Before parallelization, generation contributes only 1.5% of the summed stage times (1,577 seconds of generation versus 102,675 CPU-seconds of verification). Parallel execution narrows this imbalance, but verification still occupies 77% of the estimated end-to-end wall-clock time. Additional generation throughput alone would therefore have limited impact; reducing proof obligations, avoiding repeated verifier work, or improving verifier load balancing targets the dominant cost.